FenE to FenX Migration Data Pipeline and Vendor Extracts Reconciliation Framework

Overview

This project supports the migration of client, document, and related entity data from the Fenergo (FenE) platform to FenX.

The pipeline performs the following functions:

1. Extract migration data from the FenE database
2. Match FenE and FenX documents
3. Match FenE and FenX clients
4. Build final FenE-to-FenX migration datasets
5. Filter data to migration-approved entities
6. Validate each processing stage
7. Compare BNS migration outputs against E2X vendor extracts

The objective is to provide complete traceability from source system extraction through final reconciliation with vendor deliverables.

High-Level Pipeline

```
STEP 01
Extract Source Data
    ↓

STEP 02
Document Matching
    ↓

STEP 03
Client Matching
    ↓

STEP 04
Build Final FenE Mapped Dataset
    ↓

STEP 05
Build Final iManage Mapped Dataset
    ↓

STEP 06
Entity In Scope Filter
    ↓
```

```
STEP 07
Validation Framework
    ↓

Vendor Reconciliation
(BNS vs E2X)
```

Project Structure

Source Extract Folder

```
temp/
```

Contains intermediate data generated by the migration pipeline.

Examples:

```
fen_client_detail.csv
fen_doc.csv
fen_doc_detail.csv
fen_product.csv
fen_comment.csv
fen_contact.csv
fen_association.csv
fen_address.csv
fen_taxid.csv
```

Final Output Folder

```
output/
```

Contains final migration outputs.

Examples:

```
final_fen_to_im.csv
final_im_to_fen.csv
```

In-Scope Output Folder

```
output_in_scope/
```

Contains migration outputs restricted to approved migration scope.

Examples:

```
in_scope_fen_client_detail.csv
in_scope_fen_doc.csv
in_scope_fen_product.csv
in_scope_fen_comment.csv
in_scope_fen_contact.csv
in_scope_fen_association.csv
in_scope_fen_address.csv
in_scope_fen_taxid.csv
```

Vendor Extract Folder

```
E2X-Solution-1.1/Output/Check_Extracts/
```

Contains extract files received from E2X.

Examples:

```
LeInScope.csv
Documents.csv
Products.csv
Comments.csv
Contacts.csv
Associations.csv
Addresses.csv
TaxIdentifiers.csv
```

Validation Folder

```
validation_output/
```

Contains validation reports, logs, and reconciliation outputs.

Examples:

```
validation_log.txt

e2x_checksum.csv

e2x_client.csv
e2x_document.csv
e2x_product.csv
e2x_comment.csv
e2x_contact.csv
e2x_association.csv
e2x_address.csv
e2x_taxid.csv

e2x_detail_mapping_summary.csv
```

Pipeline Orchestration

The pipeline is orchestrated through:

```
main.py
```

Each stage is executed through a common framework:

```
run_step()
```

For each processing stage:

1. Execute pipeline logic
2. Execute validation logic
3. Update validation context
4. Log validation results
5. Stop pipeline immediately if validation fails

Benefits:

```
Consistent execution
Consistent validation
Centralized logging
Centralized error handling
Controlled pipeline flow
```

STEP 01 - Extract

Module

```
pipeline/_01_extract.py
```

Purpose

Extract migration data from the FenE SQL database and save dimensional datasets to CSV.

This is the primary source data acquisition stage.

Inputs

```
FenE SQL Database
```

Primary entities include

```
Legal Entity
Document
Product
Comment
Contact
Association
Address
Tax Identifier
```

Outputs

Location:

```
temp/
```

Files:

```
fen_client_detail.csv
fen_doc.csv
fen_doc_detail.csv
fen_product.csv
fen_comment.csv
fen_contact.csv
```

```
fen_association.csv
fen_address.csv
fen_taxid.csv
```

Validation

Validation Function:

```
validate_step_01_extract()
```

Checks:

```
File existence
Row count
Column count
Required columns
Key completeness
Null key validation
```

Required keys include:

```
LegalEntityId
DocumentId
ProductId
CommentId
ContactId
AssociatedRelationId
AddressId
TaxIdentiferId
```

STEP 02 - Match Documents

Module

```
pipeline/_02_match_documents.py
```

Purpose

Perform document-level matching between FenE and iManage by 4 ordered layers:

```
    Layer 1:
        iManage_Doc_Num = docnum

    Layer 2:
        LegalEntityId = c1alias
        AND DocumentName = docname

    Layer 3:
        ReferenceId = c1alias
        AND DocumentName = docname

    Layer 4:
        LegalEntityName = C_DESCRIPT
        AND DocumentName = docname

    Earlier layers win.
```

Creates candidate document mappings for migration.

Inputs

```
temp/fen_client_doc.csv
temp/im_client_doc.csv
```

Output

```
output/doc_fen_to_im.csv
output/doc_im_to_doc.csv
```

Processing Summary

Document records are matched using document-related attributes and identifiers.

Validation

Validation Function:

```
validate_step_02_document_match()
```

Checks:

```
Document match count
Duplicate match detection
Missing document keys
Match coverage analysis
```

STEP 03 - Match Clients

Module

```
pipeline/_03_match_clients.py
```

Purpose

Perform client-level matching using fuzzy matching techniques. Only client-level migration keys are retained.

```
  Fen client match to IM client.

    Layers:
        1. LegalEntityId = c1alias
        2. ReferenceId = c1alias
        3. LegalEntityName fuzzy C_DESCRIPT
        4. Alias1 fuzzy C_DESCRIPT
        5. Alias2 fuzzy C_DESCRIPT
        6. Alias3 fuzzy C_DESCRIPT
        7. Alias4 fuzzy C_DESCRIPT
```

Inputs

```
temp/fen_client_doc.csv
temp/im_client_doc.csv
```

Output

```
output/client_fen_to_im.csv
output/client_im_to_doc.csv
```

Processing Summary

Clients are matched using client-related attributes and identifiers. RefClientID LegalEntityId or iManage ClientId as matched reference client Id

Validation

Validation Function:

```
validate_step_03_client_match()
```

Checks:

```
Match count
Distinct client count
Duplicate client matches
Missing keys
```

STEP 04 - Final FenE Mapped Dataset

Module

```
pipeline/_04_final_fen.py
```

Purpose

Build the final FenE mapped dataset.

Inputs

```
output/doc_fen_to_im.csv
output/client_fen_to_im.csv
output/fen_association.csv
output/fen_product.csv
output/fen_case.csv
```

Processing Summary

Multiple datasets are merged onto:

```
LegalEntityId
```

One-to-many relationships are aggregated before merging to avoid row explosion.

Examples:

```
Product Counts
Association Counts
Case Counts
```

Output

```
output/final_fen_to_im.csv
```

Validation

Validation Function:

```
validate_step_04_final_fen()
```

Checks:

```
Row count consistency
Distinct LegalEntityId count
Document count preservation
Product count preservation
Association count preservation
Case count preservation
```

Additional checks:

```
Sample entity validation
Merge integrity validation
Unexpected row growth
Missing record detection
```

STEP 05 - Final iManage Mapped Dataset

Module

```
pipeline/_05_final_im.py
```

Purpose

Build reverse mapping for iManage mapped dataset:

Inputs

```
output/doc_im_to_fen.csv
output/client_im_to_fen.csv
```

Processing Summary

Multiple datasets are merged onto:

```
c1alias - iManage client ID
```

Output

```
output/final_im_to_fen.csv
```

Validation

Validation Function:

```
validate_step_05_final_im()
```

Checks:

```
Distinct entity counts
Distinct document counts
```

STEP 06 - Entity In Scope Filter

Module

```
pipeline/_06_entity_in_scope_filter.py
```

Purpose

Restrict migration outputs to approved migration entities.

Inputs

```
output/final_fen_to_im.csv
temp/fen_entity_in_scope.csv
```

Processing Logic

Keep only records where:

```
LegalEntityId
exists in
temp/fen_entity_in_scope.csv
```

Outputs

```
output_in_scope/
output_out_scope/
```

Examples:

```
in_scope_fen_client_detail.csv
in_scope_fen_doc.csv
in_scope_fen_product.csv
in_scope_fen_comment.csv
in_scope_fen_contact.csv
in_scope_fen_association.csv
in_scope_fen_address.csv
in_scope_fen_taxid.csv

out_scope_fen_client_detail.csv
out_scope_fen_doc.csv
out_scope_fen_product.csv
out_scope_fen_comment.csv
out_scope_fen_contact.csv
out_scope_fen_association.csv
```

```
out_scope_fen_address.csv
out_scope_fen_taxid.csv
```

Validation

Validation Function:

```
validate_step_06_entity_scope_filter()
```

Checks:

```
In-scope counts reconcile with source
```

STEP 07 - Validation Framework

Module

```
pipeline/_07_validation.py
```

Purpose

Provide centralized validation and reporting for all pipeline stages.

Responsibilities

Validation Logging

Output:

```
validation_output/validation_log.txt
```

Validation Reports

Generate:

```
Row count validations
Sample record validations
```

Pipeline Context Tracking

The validation framework maintains shared state between stages: This enables:

```
Count verification
Sample verification
Pipeline consistency checks
```

Vendor Reconciliation

Module

```
analysis/e2x_compare_extracts.py
```

Purpose

Compare BNS migration outputs against E2X vendor outputs.

This validation occurs after:

```
Entity In Scope Filter
```

to ensure only approved migration entities are reconciled.

Reconciliation Inputs

BNS Output

Location:

```
output_in_scope/
```

Files:

```
in_scope_fen_client_detail.csv
in_scope_fen_doc.csv
in_scope_fen_product.csv
in_scope_fen_comment.csv
in_scope_fen_contact.csv
```

```
in_scope_fen_association.csv
in_scope_fen_address.csv
in_scope_fen_taxid.csv
```

Vendor Output

Location:

```
E2X-Solution-1.1/Output/Check_Extracts/
```

Files:

```
LeInScope.csv
Documents.csv
Products.csv
Comments.csv
Contacts.csv
Associations.csv
Addresses.csv
TaxIdentifiers.csv
```

Client

```
LegalEntityId
```

Document

```
LegalEntityId + DocumentId
```

Product

```
LegalEntityId + ProductId
```

Comment

```
LegalEntityId + CommentId
```

Contact

```
LegalEntityId + ContactId
```

Association

```
LegalEntityId + AssociatedRelationId
```

Address

```
LegalEntityId + AddressId
```

Tax Identifier

```
LegalEntityId + TaxIdentiferId
```

Vendor Checksum Validation

Purpose

Compare distinct business key counts between BNS and E2X.

Output

```
validation_output/e2x_checksum.csv
```

Columns:

```
ExtractName
BNSKeyCount
E2XKeyCount
Difference
Status
```

Vendor Detail Validation

Purpose

Compare every business key individually.

Outputs

```
e2x_client.csv
e2x_document.csv
e2x_product.csv
e2x_comment.csv
e2x_contact.csv
e2x_association.csv
e2x_address.csv
e2x_taxid.csv
```

Mapping Status

Mapped

Exists in:

```
BNS and E2X
```

Only_In_BNS_Result

Exists in:

```
BNS only
```

Only_In_E2X_Result

Exists in:

```
E2X only
```

Final Deliverables

After successful execution, the following outputs should exist.

Final Mapped Datasets

```
output/

final_fen_to_im.csv
final_im_to_fen.csv
```

In-Scope Outputs

```
output_in_scope/

in_scope_fen_client_detail.csv
in_scope_fen_doc.csv
in_scope_fen_product.csv
in_scope_fen_comment.csv
in_scope_fen_contact.csv
in_scope_fen_association.csv
in_scope_fen_address.csv
in_scope_fen_taxid.csv
```

Validation Outputs

```
validation_output/

e2x_checksum.csv

e2x_client.csv
e2x_document.csv
e2x_product.csv
e2x_comment.csv
e2x_contact.csv
e2x_association.csv
e2x_address.csv
e2x_taxid.csv

e2x_detail_mapping_summary.csv
```

Execution Instructions

Run Full Migration Pipeline

From project root:

```
py -m main
```

Run Vendor Reconciliation Only

```
py -m analysis.e2x_compare_extracts
```

Success Criteria

The migration pipeline is considered successful when:

```
All pipeline steps complete successfully
All validation checks pass
Key counts reconcile between stages
Vendor checksum differences are explained
Vendor detail mapping results are reconciled
Final migration outputs are traceable back to source data
```

This framework provides complete traceability from FenE source data through migration outputs and final vendor reconciliation.